DTM — Development Traceability Module

DTM — Development Traceability Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Creation Standard (SCS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Development Traceability Governance

Module ID: SLGA-SCS-DTM-005

Origin ID: OOF.CM.SLGA.02.05.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 11.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Development Traceability Module (DTM) governs how material

relationships between software creation objectives, applicable requirements, creation actors, development methods, authority conditions, execution activities, constituent actions, tools, software artifacts, transformations, composition states, and supporting evidence are identified, established, maintained, evaluated, and reconstructed throughout software creation.

DTM establishes the methodological conditions under which a software

creation history may be recognized as sufficiently traceable to support the reconstruction of how, why, by whom or by what, under which authority, and through which material actions a software artifact came into existence.

It governs the continuity and evidentiary support of development

relationships across human-directed programming, AI-assisted development, autonomous coding agents, multi-agent systems, automated software generation, software transformation, and hybrid creation processes.

DTM distinguishes the existence of development logs from governed

traceability, recorded relationships from evidence-supported relationships, and reconstruction of a creation history from proof that every underlying action was legitimate or that the resulting software satisfies its governing requirements.

Its principal output is:

Governed Development Traceability State

Its foundational distinction is:

Recorded Development Activity ≠ Governed Development Traceability.

Operational Role

The operational role of DTM is:

To ensure that material software creation decisions, actors, authority references, execution activities, constituent actions, artifact transformations, and resulting software outputs remain connected through identifiable, evidence-supported, temporally meaningful, and reconstructable relationships across the software creation process.

DTM establishes the development traceability layer of SCS.

It connects the relevant governance states produced by the preceding SCS

Core Modules without replacing their respective responsibilities.

DTM does not independently determine that an action was authorized, that an artifact possesses sufficient integrity, or that a software product is functionally correct.

It preserves the relationships and evidence required for competent

governance processes to make those determinations.

Position in the SCS Internal Governance Flow

The SCS internal governance flow is:

Development Method Selection → Code & Artifact Integrity → Autonomous

Software Creation Control → Software Artifact Composition → Development Traceability

DTM governs:

Development Traceability

It is the fifth Core Module of the Software Creation Standard (SCS).

Its principal upstream references include:

Governed Software Requirements Baseline

from Software Requirements Standard (SRS).

Governed Development Method Selection State

from DMSM — Development Method Selection Module.

Governed Code & Artifact Integrity State

from CRIM — Code & Artifact Integrity Module.

Governed Autonomous Software Creation Control State

from ASCCM — Autonomous Software Creation Control Module, where applicable.

Governed Software Artifact Composition State

from SACM — Software Artifact Composition Module.

Its principal output is:

Governed Development Traceability State

This state contributes to the broader SCS output:

Governed Software Creation State

DTM is cross-cutting within the creation process.

Traceability relationships may be established continuously rather than reconstructed only after software development has ended.

Central Operational Question

How can the complete material history of software creation remain reconstructable across requirements, actors, authority, methods, tools, actions, transformations, and resulting artifacts without confusing recorded technical activity with evidence-supported governance traceability?

Development Traceability Governance Object

Every DTM assessment shall identify the Development Traceability Governance

Object.

The object represents a defined software creation activity, artifact

history, or material relationship set whose traceability is being governed.

It should identify:

Traceability Identity: The identifiable traceability assessment or record.

Creation Reference: The relevant software creation activity.

Creation Objective: The intended software outcome.

Requirements Reference: The applicable governed requirements basis.

Actor References: Material human, organizational, AI, or autonomous

participants.

Authority References: Relevant authorization and delegation conditions.

Method References: Applicable development methods.

Execution References: Material development activities and actions.

Artifact References: Resulting and contributing software artifacts.

Transformation References: Material artifact modification events.

Composition References: Relevant constituent and composite artifact

relationships.

Evidence References: Records supporting the claimed relationships.

Traceability State: The current governance condition of the relationship

set.

The traceability object shall remain distinguishable from any individual

record or artifact within it.

Traceability Object ≠ Individual Development Log.

Development Traceability Scope

DTM shall establish the scope of the software creation history being

assessed.

Scope may concern:

• A complete software creation activity

• A particular feature

• A software component

• A development task

• An autonomous agent execution

• A multi-agent development workflow

• An artifact transformation

• A composite software artifact

• A defined development period

• A specific creation-governance question

The required traceability depth shall reflect the materiality of the

governed activity.

DTM does not require every insignificant technical event to be preserved indefinitely.

It requires material relationships to remain sufficiently reconstructable.

Traceability Scope ≠ Unlimited Event Collection.

Development Traceability Identity

DTM shall establish identifiable references for material traceability

relationships.

Identity may be supported by:

• Traceability object identifiers

• Creation activity references

• Requirements identifiers

• Actor identifiers

• Execution-event identifiers

• Artifact identifiers

• Transformation identifiers

• Composition identifiers

• Evidence references

A relationship identifier shall not be treated as sufficient proof that the relationship is accurate.

Traceability Identifier ≠ Verified Traceability Relationship.

Traceability Relationship Classification

DTM shall classify material development relationships according to their

governance meaning.

Relevant relationship classes may include:

Origin Relationship: Connects a creation object or artifact to its

supported origin.

Purpose Relationship: Connects an activity to its intended objective.

Requirements Relationship: Connects creation activities or artifacts to

applicable requirements.

Authority Relationship: Connects material actions to their authority

references.

Actor Relationship: Connects activities to the actors or mechanisms

performing them.

Method Relationship: Connects execution to the recognized development

method.

Action Relationship: Connects constituent actions to the broader creation

activity.

Tool-Path Relationship: Connects actions to the tools and paths used.

Transformation Relationship: Connects source and resulting artifact

states.

Composition Relationship: Connects constituent artifacts to composite

outputs.

Evidence Relationship: Connects claims to supporting evidence.

Continuity Relationship: Connects historical creation states across

change.

These classes shall remain distinguishable where their evidentiary meaning differs.

Recorded Association ≠ Governed Relationship.

Creation Objective Traceability

DTM shall preserve the relationship between material development activities

and the intended software creation objective.

Relevant questions include:

• Which objective initiated the activity?

• Which material actions were undertaken to achieve it?

• Which artifacts resulted?

• Did the objective change?

• Were new objectives introduced?

• Which evidence supports the relationships?

An activity may be technically related to a project without being connected to the authorized creation objective.

Project Association ≠ Creation Objective Traceability.

Requirements-to-Creation Traceability

DTM shall preserve material relationships between governed requirements and

software creation activities.

Relevant references may include:

• Requirements baseline

• Applicable requirement identity

• Creation task

• Implementation activity

• Resulting artifact

• Relevant verification reference

• Requirements change history

DTM shall distinguish a recorded requirements link from evidence that the artifact actually satisfies the requirement.

Requirement-to-Code Link ≠ Requirement Satisfaction.

Detailed requirements traceability governance remains within:

RTM — Requirements Traceability Module

under Software Requirements Standard (SRS).

DTM governs the creation-side reconstruction of those relationships.

Creation Authority Traceability

DTM shall preserve material relationships between software creation actions

and their applicable authority references.

These may include:

• Creation authorization

• Delegated execution authority

• Permitted action categories

• Tool permissions

• Artifact modification authority

• Agent delegation

• Relevant approval events

The existence of an authority reference does not independently establish that the reference was valid or applicable.

Authority Reference ≠ Established Action Authorization.

DTM preserves the relationship for competent authority assessment.

Actor Traceability

DTM shall identify material actors involved in software creation.

Actors may include:

• Human developers

• Development organizations

• External suppliers

• AI coding assistants

• Autonomous agents

• Multi-agent systems

• Automated development tools

• Build and generation mechanisms

Traceability shall distinguish actor identity from the actor's role and

authority.

A recorded username may not establish the identity of the actual human or autonomous mechanism responsible for an action.

Recorded User Identity ≠ Established Execution Actor.

Human Development Traceability

DTM applies to human-directed software creation.

Relevant relationships may include:

• Assigned development tasks

• Implementation decisions

• Source-code modifications

• Review activities

• Artifact creation

• Relevant approvals

• Material exceptions

Human participation does not eliminate the need for traceability.

A developer's recollection may support reconstruction but shall not automatically substitute for sufficient contemporaneous evidence.

Human Testimony ≠ Complete Development History by default.

AI-Assisted Development Traceability

DTM applies where AI systems contribute to software creation under human direction.

Relevant traceability relationships may include:

• Human creation objective

• AI-assisted task

• Material input context

• Generation mechanism

• Generated output

• Human modification

• Accepted artifact state

• Supporting evidence

DTM shall distinguish AI-generated content from human-authored content

where the distinction is material and sufficiently supported.

AI Assistance Record ≠ Complete AI Contribution Traceability.

Autonomous Development Traceability

DTM applies to autonomous software creation.

Relevant relationships may include:

• Delegated objective

• Autonomous actor identity

• Execution plan

• Constituent actions

• Tool invocations

• Data access

• Artifact modifications

• Exceptions

• Interventions

• Resulting artifacts

An autonomous system may complete a task while its material execution path remains insufficiently reconstructable.

Autonomous Task Completion ≠ Autonomous Execution Traceability.

Detailed autonomous execution control remains within ASCCM.

Multi-Agent Development Traceability

DTM shall preserve material relationships across multiple agents

contributing to software creation.

These may include:

• Agent identities

• Agent roles

• Task delegation

• Handoff events

• Shared context

• Tool actions

• Artifact contributions

• Integration decisions

• Resulting outputs

Traceability shall preserve material authority and execution continuity

across agent boundaries.

Agent Handoff Record ≠ Complete Delegation Traceability.

Development Method Traceability

DTM shall preserve the relationship between creation activities and the

applicable development method.

Its principal reference is:

Governed Development Method Selection State

from DMSM.

Relevant information may include:

• Selected method

• Method applicability

• Material method conditions

• Actual execution approach

• Method deviations

• Method changes

• Reassessment references

A method may be documented while the actual creation process materially differs from it.

Documented Method ≠ Executed Method.

Development Decision Traceability

DTM shall preserve material development decisions where they affect the creation outcome or its governance interpretation.

Relevant decisions may include:

• Implementation approach

• Component selection

• Tool selection

• Architecture-related implementation choices

• Artifact transformation

• Scope adaptation

• Integration strategy

• Exception handling

Decision traceability should identify:

Decision Context → Decision Actor → Decision Basis → Resulting Action or

Artifact

A recorded decision shall not automatically be treated as justified or

authorized.

Recorded Decision ≠ Governed Decision.

Constituent Action Traceability

DTM shall preserve material relationships between a software creation

objective and the actions performed to achieve it.

A relevant chain may be:

Creation Objective → Delegated Authority → Constituent Action → Tool

Execution → Resulting Artifact State

Constituent actions may include:

• File access

• Code generation

• Repository modification

• Package installation

• Command execution

• Data retrieval

• Test execution

• Artifact transformation

• Agent delegation

DTM shall preserve relevant action identity, actor, context, and evidence.

Task Traceability ≠ Constituent Action Traceability.

Tool-Path Traceability

DTM shall preserve material relationships between development actions and

the tool paths used.

A tool path may include:

Agent → Tool Interface → External Service → Data Source → Execution Result

Relevant evidence may concern:

• Tool identity

• Invocation context

• Actor

• Permission reference

• Material input

• Resulting output

• Downstream action

• Execution outcome

A single successful tool invocation does not establish the legitimacy of the complete path.

Tool Invocation Success ≠ Governed Tool-Path Validity.

Data and Context Traceability

Software creation may depend on material information inputs.

These may include:

• Requirements documents

• Source-code context

• Retrieved documentation

• External data

• Internal databases

• Agent memory

• Shared development context

• Generated intermediate information

DTM shall preserve material relationships between such inputs and the

resulting creation activities where necessary for reconstruction.

It does not independently authorize data access or determine the legitimacy of every information source.

Data Usage Record ≠ Authorized Data Use.

Software Artifact Traceability

DTM shall preserve material relationships between creation activities and

resulting software artifacts.

Relevant references may include:

• Artifact identity

• Artifact version

• Creation event

• Contributing actor

• Development method

• Material inputs

• Transformation history

• Composition references

• Supporting evidence

DTM consumes relevant integrity information from CRIM.

It does not independently establish complete artifact integrity.

Artifact Link ≠ Artifact Integrity Proof.

Artifact Transformation Traceability

DTM shall preserve material relationships across software artifact

transformations.

The traceability chain is:

Source Artifact State → Transformation Event → Transformation Actor or

Mechanism → Resulting Artifact State

Transformations may include:

• Editing

• Refactoring

• Regeneration

• Compilation

• Code translation

• Optimization

• Automated rewriting

• Packaging

A transformation record shall remain connected to the relevant source and

resulting artifact identities.

Transformation Log ≠ Verified Transformation History.

Artifact Composition Traceability

DTM shall preserve material relationships between constituent artifacts and

resulting composite artifacts.

Its principal reference is:

Governed Software Artifact Composition State

from SACM.

Relevant relationships may include:

• Constituent artifact identity

• Constituent version

• Composition role

• Integration mechanism

• Composition transformation

• Resulting composite identity

• Supporting evidence

A composition manifest may contribute to traceability but does not independently establish the complete composition history.

Composition Manifest ≠ Complete Composition Traceability.

Generated Artifact Lineage

DTM shall preserve material lineage relationships for generated software

artifacts.

A relevant chain may be:

Generation Objective → Material Inputs → Generation Mechanism → Generated

Artifact → Subsequent Transformation

The required level of detail depends on materiality and available evidence.

DTM shall distinguish recorded generation facts from inferred explanations.

Generated Artifact Explanation ≠ Verified Artifact Lineage.

Development Event Identity

DTM shall identify material development events sufficiently to distinguish

them from other events.

Relevant event information may include:

• Event identifier

• Event type

• Actor reference

• Associated objective

• Relevant tool

• Artifact reference

• Temporal context

• Supporting evidence

Event identity shall not be confused with event validity.

Recorded Event ≠ Governed Event.

Temporal Traceability

DTM shall preserve relevant temporal relationships between material

development events.

These may include:

• Event occurrence time

• Event ordering

• Relevant version state

• Authority validity period

• Requirements applicability period

• Transformation sequence

• Agent handoff sequence

• Evidence creation time

Temporal ordering may be uncertain in distributed development systems.

DTM shall preserve that uncertainty rather than invent a definitive sequence.

Timestamp Availability ≠ Verified Event Order.

Distributed Development Traceability

Software creation may occur across multiple repositories, systems, organizations, and execution environments.

DTM shall preserve material cross-environment relationships.

Relevant conditions may include:

• Actor identity continuity

• Event correlation

• Artifact identity continuity

• Tool-path references

• Distributed timestamps

• Evidence source differences

• Repository transitions

• Organizational handoffs

A common project identifier does not independently establish complete cross-system traceability.

Shared Project Identifier ≠ Distributed Traceability Continuity.

Traceability Evidence

DTM shall preserve evidence supporting material development relationships.

Evidence may include:

• Requirements records

• Development decisions

• Authority references

• Repository history

• Tool execution records

• Agent activity records

• Artifact identifiers

• Transformation records

• Composition manifests

• Review records

• Build records

• Relevant system logs

Evidence requirements shall reflect the materiality and intended governance

use of the traceability relationship.

Log Availability ≠ Traceability Evidence Sufficiency.

Traceability Evidence Competence

DTM shall assess whether evidence sources are competent to support the specific relationships claimed.

For example:

A repository record may support a recorded modification event.

A tool log may support an invocation claim.

An artifact hash may support content correspondence.

A requirements record may support the existence of a requirement reference.

An agent-generated explanation may support a proposed reconstruction hypothesis.

These sources do not establish identical facts.

Evidence Source Availability ≠ Evidence Source Competence.

Traceability Evidence Integrity

DTM shall preserve the integrity of evidence supporting material

traceability claims.

Relevant considerations may include:

• Evidence identity

• Source

• Content correspondence

• Temporal relevance

• Preservation history

• Transformation

• Access restrictions

• Material contradictions

• Missing records

Evidence integrity does not independently establish the correctness of the claim that the evidence is used to support.

Evidence Integrity ≠ Claim Validity.

Recorded and Inferred Relationships

DTM shall distinguish directly supported relationships from inferred

relationships.

For example:

A build record may directly identify a source version used during compilation.

An analytical system may infer that a particular generated function originated from a prior agent instruction.

Both may be useful.

They do not possess the same evidentiary status.

Inferred Relationship ≠ Established Relationship.

Where inference is used, DTM shall preserve the inference basis, uncertainty, and relevant limitations.

Traceability Completeness

DTM shall assess whether material development relationships are sufficiently represented for the intended governance purpose.

Completeness may concern:

• Objective coverage

• Requirements references

• Actor identification

• Authority relationships

• Material actions

• Tool paths

• Artifact transformations

• Composition relationships

• Evidence support

• Temporal continuity

Completeness shall be assessed relative to defined scope and materiality.

Numerous Traceability Links ≠ Complete Traceability.

Traceability Consistency

DTM shall assess whether material traceability relationships are mutually consistent.

Possible inconsistencies include:

• Contradictory actor attribution

• Conflicting artifact versions

• Incompatible event ordering

• Requirements references outside their applicable period

• Tool actions inconsistent with the recorded method

• Composition records inconsistent with resulting artifacts

Consistency does not independently establish completeness or truth.

Traceability Consistency ≠ Traceability Completeness.

Traceability Consistency ≠ Historical Accuracy.

Traceability Coverage

DTM shall identify which material portions of the software creation process

are supported by traceability evidence.

Coverage may concern:

• Creation objectives

• Requirements

• Actors

• Actions

• Tools

• Artifacts

• Transformations

• Composition

• Evidence

• Continuity

Coverage may be represented qualitatively or quantitatively where appropriate.

A numerical coverage percentage shall not automatically be interpreted as

governance sufficiency.

Traceability Coverage Percentage ≠ Traceability Governance Adequacy.

Traceability Gap

A Development Traceability Gap exists where a material creation relationship cannot be sufficiently established.

Examples include:

• An artifact with no supported creation actor

• A code modification without a reconstructable transformation event

• An autonomous tool action without a relevant authority reference

• A generated component without sufficient input lineage

• A composite artifact with missing constituent references

• A requirements implementation claim without supporting creation

evidence.

DTM shall identify the gap and its material significance.

Missing Traceability ≠ Proven Misconduct.

Critical Traceability Gap

A Critical Development Traceability Gap exists where missing or contradictory evidence prevents reconstruction of a materially consequential software creation relationship.

Examples may include:

• A critical software component with unknown modification history

• An autonomous agent execution involving untraceable privileged tool

• A production-related artifact with no reliable source-to-output

• A safety-relevant software transformation whose actor and method

calls. linkage. cannot be established.

DTM shall preserve the critical gap and support competent escalation.

It does not independently determine legal responsibility or operational suspension.

Critical Traceability Gap Identification ≠ Liability Determination.

Traceability Conflict

Traceability conflicts arise when material records support incompatible accounts of a development relationship.

Examples include:

• Different actors attributed to the same event

• Contradictory artifact versions

• Inconsistent event timestamps

• Conflicting authority references

• Incompatible composition records

• Disagreement between tool logs and repository history

DTM shall preserve the conflicting evidence.

It shall not silently resolve contradictions by selecting the most

convenient record.

Conflicting Traceability Evidence ≠ Permission to Invent Continuity.

Traceability Uncertainty

Development traceability may remain uncertain due to:

• Missing historical records

• Incomplete tool logs

• Distributed system timing

• Unavailable AI generation context

• Ambiguous actor attribution

• Incomplete transformation evidence

• Reconstructed rather than original records

• Conflicting source information

DTM shall preserve material uncertainty.

Uncertainty may affect specific relationships without invalidating the entire traceability object.

Partial Traceability Uncertainty ≠ Universal Traceability Failure.

Traceability Drift

Traceability Drift occurs when recorded relationships no longer adequately represent the actual software creation state.

Possible causes include:

• Unrecorded code changes

• Agent-generated modifications

• Requirements changes

• Method changes

• Tool-path changes

• Artifact replacement

• Composition changes

• Repository migration

• Loss of evidence continuity

DTM shall identify material drift and support reassessment.

Previously Established Traceability ≠ Permanently Established

Traceability.

Traceability Change Governance

DTM shall preserve material changes to traceability relationships.

These may include:

• Added relationships

• Removed relationships

• Corrected attribution

• Updated artifact references

• Superseded requirements links

• New evidence

• Revised reconstruction

• Resolved conflicts

Changes shall preserve the distinction between historical facts and later

corrections.

A later correction shall not silently erase the original recorded state

where that history remains material.

Traceability Correction ≠ Historical Record Erasure.

Traceability Reconstruction

DTM shall support reconstruction of material software creation histories.

Reconstruction may involve combining:

• Repository records

• Requirements history

• Agent execution evidence

• Tool logs

• Artifact provenance

• Composition manifests

• Development decisions

• Human testimony

• Other competent evidence

Reconstruction shall distinguish:

Directly Evidenced History

from:

Inferred Historical Relationships

and:

Unresolved Historical Gaps

A coherent reconstructed narrative shall not automatically be treated as a

verified history.

Plausible Reconstruction ≠ Verified Reconstruction.

Traceability Governance States

DTM shall support identifiable traceability governance states.

Possible states include:

Identified: The traceability object has been established.

Under Assessment: Material relationships are being evaluated.

Traceability-Supported: Relevant relationships are sufficiently evidenced.

Conditionally Supported: Recognition depends on explicit conditions.

Partially Supported: Some material relationships remain insufficiently

established.

Evidence-Limited: Available evidence is insufficient.

Origin-Unresolved: Material origin relationships remain unsupported.

Actor-Unresolved: Material actor attribution remains uncertain.

Authority-Unresolved: Relevant authority relationships cannot be

sufficiently established.

Transformation-Unresolved: Material artifact transformation history

remains incomplete.

Conflicted: Material traceability evidence contradicts itself.

Traceability-Failed: A material traceability condition has been

determined not to hold.

Superseded: A later governed traceability state has replaced the earlier

state.

Unresolved: Available information does not support a sufficiently determined state.

Traceability states shall not automatically be interpreted as

authorization, quality, or compliance determinations.

Traceability-Supported ≠ Creation Legitimacy Established.

Traceability Recognition

DTM shall determine whether material development relationships are sufficiently supported for their intended governance use.

Recognition should consider:

• Traceability scope

• Relationship identity

• Evidence competence

• Evidence integrity

• Materiality

• Coverage

• Completeness

• Consistency

• Temporal relevance

• Uncertainty

• Conflicts

• Continuity

Recognition shall remain limited to the relationships actually assessed.

Recognized Traceability ≠ Universal Creation Acceptance.

Traceability and Accountability

DTM may preserve evidence relevant to identifying actors, decisions, actions, and outcomes.

Such evidence may support competent accountability assessments.

However, traceability alone does not establish:

• Legal liability

• Organizational responsibility

• Fault

• Negligence

• Authorized conduct

• Contractual responsibility

• Causal responsibility

These determinations require the applicable competent governance and legal

processes.

Action Attribution ≠ Accountability Determination.

Traceability Evidence ≠ Liability Proof.

Traceability and Software Quality

DTM may connect software artifacts to requirements, implementation activities, and verification evidence.

However, the existence of these relationships does not establish functional correctness.

A software artifact may be completely traceable and still contain defects.

A requirement may be linked to a test that fails.

A passing test may cover only part of the relevant requirement.

Traceable Software ≠ Correct Software.

Test Link ≠ Verified Requirement Satisfaction.

Detailed software quality governance remains within Software Quality Standard (SQS).

Traceability and Software Security

DTM may preserve evidence relevant to security assessment, including:

• Source-code changes

• Dependency introductions

• Tool actions

• Artifact transformations

• Relevant actor attribution

• Security review references

However, traceability does not independently establish security.

A fully traceable software component may still contain vulnerabilities.

Complete Creation History ≠ Secure Software.

Detailed security governance remains within Software Security Standard (SSS).

Traceability and Operational Reality

DTM shall distinguish recorded development activity from the actual

operational conditions under which the activity occurred.

Technical records may omit:

• Unrecorded human intervention

• External system behavior

• Undocumented environmental changes

• Tool failures

• Missing data access

• Unobserved execution conditions

Where operational reality cannot be sufficiently reconstructed, the limitation shall remain explicit.

Recorded Development Reality ≠ Complete Operational Reality.

Competent operational reality governance remains within ORA™.

Traceability Continuity

DTM shall preserve material creation relationships across changes in:

• Human developers

• AI agents

• Development tools

• Repositories

• Software versions

• Requirements baselines

• Organizations

• Development environments

• Generation mechanisms

• Artifact composition

Continuity shall be supported by identifiable relationships and sufficient

evidence.

It shall not be inferred merely from the persistence of a project name or

repository.

Project Continuity ≠ Development Traceability Continuity.

Traceability Preservation

DTM shall preserve material traceability evidence for the periods and

purposes established by applicable governance conditions.

Preservation may require:

• Stable identifiers

• Historical version references

• Evidence integrity

• Relationship history

• Relevant temporal context

• Controlled corrections

• Supersession records

• Accessible reconstruction references

DTM does not establish universal legal retention periods.

Evidence preservation shall remain compatible with competent privacy,

security, retention, and deletion obligations.

Evidence Preservation ≠ Unlimited Data Retention Authority.

Traceability Reassessment

DTM shall support reassessment when material creation conditions change.

Triggers may include:

• New evidence

• Requirements changes

• Artifact modifications

• Autonomous execution changes

• New actor attribution

• Tool-path changes

• Composition changes

• Integrity conflicts

• Repository migration

• Evidence loss

• Traceability drift

Reassessment shall preserve historical traceability states where necessary for reconstruction.

Governed Development Traceability State

The principal output of DTM is: Governed Development Traceability State

The state shall preserve sufficient information to identify:

Traceability Governance Object: The creation history or relationship set

under governance.

Traceability Identity: The relevant traceability reference.

Creation Objective: The intended software outcome.

Requirements References: The applicable governed requirements.

Creation Actors: Relevant human, organizational, AI, and autonomous

participants.

Actor Roles: The material roles performed.

Authority References: Applicable creation and execution authority

relationships.

Development Method References: The methods associated with creation.

Development Decisions: Material implementation and transformation

decisions.

Execution Activities: Relevant development events.

Constituent Actions: Material actions performed to achieve the objective.

Tool Paths: Relevant tool-selection and execution relationships.

Data and Context References: Material information inputs.

Artifact References: Relevant software artifacts and versions.

Transformation Relationships: Source-to-result artifact relationships.

Composition Relationships: Constituent-to-composite artifact

relationships.

Generated Artifact Lineage: Relevant generation relationships.

Temporal Relationships: Material event timing and ordering.

Evidence References: Records supporting traceability claims.

Evidence Integrity: The condition of supporting evidence.

Relationship Confidence: The evidentiary status and uncertainty of

material relationships.

Coverage: The material creation relationships represented.

Completeness: The assessed sufficiency of the traceability scope.

Consistency: The compatibility of material relationship records.

Traceability Gaps: Missing material relationships.

Critical Traceability Gaps: Material reconstruction failures requiring

competent attention.

Conflicts: Contradictory traceability evidence.

Uncertainty: Material unresolved historical relationships.

Traceability Drift: Divergence from the recognized creation history.

Traceability Governance State: The current recognized traceability

condition.

Continuity: Historical relationship preservation.

Downstream References: Relevant information for competent software

lifecycle governance.

The state shall not independently establish creation authorization,

software correctness, security, accountability, liability, release readiness, or deployment authorization.

Minimum Implementation Framework

A conforming implementation of DTM must provide the following six functional capabilities.

• Development Traceability Object and Relationship Governance

Capability to identify traceability objects, define material relationship

types, establish creation scope, and preserve stable references across objectives, requirements, actors, methods, actions, tools, and artifacts.

The implementation must distinguish relationship identifiers from evidence-supported relationship claims.

• Actor, Authority, Method, and Constituent Action Traceability Governance

Capability to preserve material relationships connecting creation actors,

authority references, development methods, execution decisions, constituent actions, and tool paths.

The implementation must not infer authorization solely from recorded action execution.

• Artifact, Transformation, Composition, and Temporal Traceability

Governance

Capability to reconstruct material relationships between source artifacts,

transformations, generated outputs, constituent artifacts, composite software, and relevant event ordering.

The implementation must preserve uncertainty where temporal or historical relationships cannot be sufficiently established.

• Traceability Evidence, Coverage, and Sufficiency Governance

Capability to associate material relationship claims with competent

evidence, evaluate evidence integrity, assess coverage, and distinguish completeness from consistency.

The implementation must not treat high log volume or numerous links as proof of traceability adequacy.

• Traceability Gap, Conflict, Drift, and Reconstruction Governance

Capability to identify missing relationships, critical gaps, contradictory

evidence, material drift, and historical reconstruction limitations.

The implementation must distinguish directly evidenced history from inferred reconstruction and unresolved gaps.

• Governed Development Traceability State and Lifecycle Handoff Governance

Capability to produce a Governed Development Traceability State with

sufficient relationship, evidence, uncertainty, continuity, and governance-state information for competent SCS creation-state recognition and downstream SLGA™ lifecycle governance.

The implementation must preserve material unresolved conditions during handoff.

GIE™ Compatibility

DTM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Development traceability object identification

• Creation relationship classification

• Requirements-to-creation mapping

• Actor attribution analysis

• Authority relationship mapping

• Development method relationship analysis

• Constituent action reconstruction

• Tool-path mapping

• Data and context lineage analysis

• Artifact lineage reconstruction

• Transformation history analysis

• Composition relationship mapping

• Temporal event correlation

• Distributed development relationship analysis

• Evidence source association

• Evidence competence assessment

• Traceability coverage analysis

• Traceability completeness assessment

• Traceability consistency assessment

• Traceability gap detection

• Critical traceability gap identification

• Traceability conflict analysis

• Traceability drift detection

• Historical creation reconstruction

• Traceability continuity assessment

• Governed traceability state representation

GIE™ may infer candidate relationships, propose reconstruction paths, or identify suspected evidence gaps.

Such outputs shall remain distinguishable from directly evidenced

relationships and competent governance determinations.

GIE™ must not fabricate missing historical events, convert plausible reconstructions into verified facts, or infer creation authorization from the existence of execution records.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical DTM governance logic is:

Receive Governed Software Requirements Baseline

↓

Receive Governed Development Method Selection State

↓

Receive Relevant Code & Artifact Integrity States

↓

Receive Relevant Autonomous Software Creation Control States

↓

Receive Governed Software Artifact Composition State

↓

Identify Development Traceability Governance Object

↓

Establish Traceability Scope

↓

Identify Material Creation Objectives

↓

Identify Applicable Requirements References

↓

Identify Creation Actors and Roles

↓

Identify Authority and Delegation References

↓

Identify Development Methods

↓

Identify Material Development Decisions

↓

Identify Execution Activities and Constituent Actions

↓

Identify Tool Paths and Data Context

↓

Identify Software Artifacts and Versions

↓

Identify Artifact Transformation Relationships

↓

Identify Composition Relationships

↓

Establish Material Temporal Relationships

↓

Associate Traceability Claims With Evidence

↓

Assess Evidence Competence and Integrity

↓

Assess Traceability Coverage

↓

Assess Traceability Completeness and Consistency

↓

Identify Traceability Gaps and Critical Gaps

↓

Identify Evidence Conflicts and Uncertainty

↓

Identify Traceability Drift

↓

Reconstruct Material Development History

↓

Determine Development Traceability Governance State

↓

Establish Governed Development Traceability State

↓

Preserve Traceability Continuity

↓

Handoff to SCS Creation-State Recognition and Competent SLGA™ Lifecycle

Governance

↓

Reassess When Material Creation Relationships Change

The principal output is:

Governed Development Traceability State

Governance Outputs

DTM may produce:

Development Traceability Governance Object

Development Traceability Scope Record

Traceability Relationship Identity Record

Traceability Relationship Classification

Creation Objective Traceability Record

Requirements-to-Creation Traceability Record

Creation Authority Traceability Record

Actor Traceability Record

Human Development Traceability Record

AI-Assisted Development Traceability Record

Autonomous Development Traceability Record

Multi-Agent Development Traceability Record

Development Method Traceability Record

Development Decision Traceability Record

Constituent Action Traceability Record

Tool-Path Traceability Record

Data and Context Traceability Record

Software Artifact Traceability Record

Artifact Transformation Traceability Record

Artifact Composition Traceability Record

Generated Artifact Lineage Record

Development Event Identity Record

Temporal Traceability Record

Distributed Development Traceability Record

Traceability Evidence Record

Evidence Source Competence Assessment

Traceability Evidence Integrity Assessment

Recorded and Inferred Relationship Classification

Traceability Completeness Assessment

Traceability Consistency Assessment

Traceability Coverage Assessment

Development Traceability Gap

Critical Development Traceability Gap

Traceability Conflict Record

Traceability Uncertainty State

Traceability Drift Record

Traceability Change Record

Development History Reconstruction

Traceability Governance State

Traceability Recognition Record

Accountability Evidence Reference

Software Quality Evidence Reference

Software Security Evidence Reference

Operational Reality Traceability Reference

Traceability Continuity Record

Traceability Preservation Record

Traceability Reassessment Record

SCS Creation-State Handoff Record

The principal output is:

Governed Development Traceability State

Critical Distinctions

DTM preserves the following distinctions:

Recorded Development Activity ≠ Governed Development Traceability

Traceability Object ≠ Individual Development Log

Traceability Scope ≠ Unlimited Event Collection

Traceability Identifier ≠ Verified Traceability Relationship

Recorded Association ≠ Governed Relationship

Project Association ≠ Creation Objective Traceability

Requirement-to-Code Link ≠ Requirement Satisfaction

Authority Reference ≠ Established Action Authorization

Recorded User Identity ≠ Established Execution Actor

Human Testimony ≠ Complete Development History by default

AI Assistance Record ≠ Complete AI Contribution Traceability

Autonomous Task Completion ≠ Autonomous Execution Traceability

Agent Handoff Record ≠ Complete Delegation Traceability

Documented Method ≠ Executed Method

Recorded Decision ≠ Governed Decision

Task Traceability ≠ Constituent Action Traceability

Tool Invocation Success ≠ Governed Tool-Path Validity

Data Usage Record ≠ Authorized Data Use

Artifact Link ≠ Artifact Integrity Proof

Transformation Log ≠ Verified Transformation History

Composition Manifest ≠ Complete Composition Traceability

Generated Artifact Explanation ≠ Verified Artifact Lineage

Recorded Event ≠ Governed Event

Timestamp Availability ≠ Verified Event Order

Shared Project Identifier ≠ Distributed Traceability Continuity

Log Availability ≠ Traceability Evidence Sufficiency

Evidence Source Availability ≠ Evidence Source Competence

Evidence Integrity ≠ Claim Validity

Inferred Relationship ≠ Established Relationship

Numerous Traceability Links ≠ Complete Traceability

Traceability Consistency ≠ Traceability Completeness

Traceability Consistency ≠ Historical Accuracy

Traceability Coverage Percentage ≠ Traceability Governance Adequacy

Missing Traceability ≠ Proven Misconduct

Critical Traceability Gap Identification ≠ Liability Determination

Conflicting Traceability Evidence ≠ Permission to Invent Continuity

Partial Traceability Uncertainty ≠ Universal Traceability Failure

Previously Established Traceability ≠ Permanently Established Traceability

Traceability Correction ≠ Historical Record Erasure

Plausible Reconstruction ≠ Verified Reconstruction

Traceability-Supported ≠ Creation Legitimacy Established

Recognized Traceability ≠ Universal Creation Acceptance

Action Attribution ≠ Accountability Determination

Traceability Evidence ≠ Liability Proof

Traceable Software ≠ Correct Software

Test Link ≠ Verified Requirement Satisfaction

Complete Creation History ≠ Secure Software

Recorded Development Reality ≠ Complete Operational Reality

Project Continuity ≠ Development Traceability Continuity

Evidence Preservation ≠ Unlimited Data Retention Authority

Cross-Architecture Boundary

DTM governs development traceability within SCS and SLGA™.

It does not independently establish general authority, artifact integrity, autonomous execution conformance, software quality, security, or liability.

Within SCS:

DMSM — Development Method Selection Module governs development method

selection and suitability.

CRIM — Code & Artifact Integrity Module governs artifact identity,

origin, transformation, and integrity.

ASCCM — Autonomous Software Creation Control Module governs autonomous

execution authority, permissions, and intervention.

SACM — Software Artifact Composition Module governs constituent and

composite artifact relationships.

DTM — Development Traceability Module governs the evidence-supported

reconstruction of material creation relationships across these domains.

Within SLGA™:

Software Requirements Standard (SRS) governs requirements formation,

applicability, change, and requirements traceability.

Software Dependency Standard (SDS) governs dependencies.

Software Configuration Standard (SCFS) governs configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs release decisions.

Software Deployment Standard (SDPS) governs deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

changes.

Software Retirement Standard (SRTS) governs retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority and delegation

governance.

AGA™ — Accountability Governance Architecture: Responsibility and

accountability relationships.

ASGA™ — Autonomous Systems Governance Architecture: Autonomous execution

governance.

AGI™ — Agent Governance Infrastructure: Agent identity, permission,

delegation, and tool-path governance.

OBIDENITY® — Origin-Bound Identity Governance Architecture: Origin,

identity, and provenance relationships.

ORA™ — Operational Reality Architecture: Competent operational reality

assessment.

MGIA™ — Memory Governance Architecture: Relevant memory provenance and

continuity governance.

DTM preserves traceability relationships with these domains without

replacing their competent governance responsibilities.

Canonical Boundary

DTM begins when a software creation activity requires evidence-supported reconstruction of material relationships between its objectives, requirements, actors, authority, methods, actions, tools, artifacts, transformations, and resulting outputs.

DTM governs:

• Development traceability objects

• Traceability scope

• Traceability identity

- Relationship classification
- Creation objective traceability
- Requirements-to-creation traceability
- Authority traceability
- Actor traceability
- Human development traceability
- AI-assisted development traceability
- Autonomous development traceability
- Multi-agent development traceability
- Development method traceability
- Development decision traceability
- Constituent action traceability
- Tool-path traceability
- Data and context traceability
- Software artifact traceability
- Transformation traceability
- Composition traceability
- Generated artifact lineage
- Development event identity
- Temporal traceability
- Distributed development traceability
- Traceability evidence
- Evidence competence
- Evidence integrity
- Recorded and inferred relationships
- Traceability completeness
- Traceability consistency
- Traceability coverage
- Traceability gaps
- Critical traceability gaps
- Traceability conflicts
- Traceability uncertainty
- Traceability drift
- Traceability changes
- Historical reconstruction
- Traceability governance states

• Traceability recognition

• Accountability evidence references

• Software quality evidence references

• Software security evidence references

• Operational reality traceability references

• Traceability continuity

• Evidence preservation

• Traceability reassessment

DTM does not independently determine:

• General legal or organizational authority

• Requirements legitimacy

• Development method suitability

• Complete artifact integrity

• Autonomous execution authorization

• Software composition validity

• Software functional correctness

• Software security conformance

• Legal liability

• Release readiness

• Deployment authorization

DTM ends when the relevant development history has been represented as a:

Governed Development Traceability State

with sufficient relationship identity, evidence, temporal context,

coverage, uncertainty, and continuity for competent SCS creation-state recognition and downstream SLGA™ lifecycle governance.

Use Case 1 — An AI Coding Agent Produces Working Software Without a

Reconstructable Tool Path

An organization authorizes an AI coding agent to create a new internal

software feature.

The agent generates code, executes commands, retrieves external

information, modifies repository files, and runs tests.

The resulting feature works.

The development platform records the final code and a summary of the

agent's activities.

However, the organization cannot sufficiently reconstruct:

• Which tools were invoked

• Which external data sources were accessed

• Which actions modified the software

• Whether all constituent actions remained within permitted boundaries

• Which material generation inputs contributed to the final artifact

• Which execution evidence supports the agent's summary

DTM evaluates:

Creation Objective → Authority → Agent → Constituent Actions → Tool Paths

→ Artifact Transformations → Resulting Software

The assessment identifies material gaps in the execution and artifact

lineage.

The final artifact may be technically functional.

Its development traceability state may nevertheless be:

Evidence-Limited

or:

Partially Supported

until sufficient relationships can be established.

Autonomous Task Completion ≠ Autonomous Execution Traceability.

Tool Invocation Success ≠ Governed Tool-Path Validity.

AI-Generated Explanation ≠ Verified Development History.

DTM prevents successful software generation from concealing an

insufficiently reconstructable creation process.

Use Case 2 — A Software Supplier Delivers a Verified Application but

Cannot Establish Its Development History

An organization purchases a software application from an external supplier.

The supplier delivers source code, executable packages, and technical

documentation.

The application passes its initial tests.

The supplier also provides a composition manifest and cryptographic hashes.

However, the organization cannot sufficiently establish:

• Which requirements baseline governed development

• Which actors created material software components

• Which development methods were used

• Which source versions produced the executable package

• Which material transformations occurred

• Whether autonomous agents participated

• Which authority conditions applied to material creation actions

DTM evaluates:

Requirements → Creation Objective → Actors → Methods → Actions → Artifacts

→ Composition → Evidence

The assessment identifies that artifact integrity evidence and technical functionality do not independently establish a reconstructable creation history.

The development traceability state may remain:

Evidence-Limited

or:

Origin-Unresolved

depending on the affected relationships.

Artifact Integrity ≠ Complete Development Traceability.

Composition Manifest ≠ Complete Composition Traceability.

Traceable Software ≠ Correct Software.

Technical Delivery ≠ Governed Creation History.

DTM ensures that external software delivery does not replace the need for evidence-supported reconstruction of material development relationships.

Canonical Closing Statement

Software creation increasingly occurs across distributed teams, automated

development environments, AI coding assistants, autonomous agents, and multi-agent systems.

A single software artifact may result from numerous decisions, tool invocations, generated outputs, transformations, integrations, and delegated actions.

The final code may be functional.

Its artifact identity may be preserved.

Its constituent components may be identifiable.

Yet the relationships explaining how the software came into existence may remain incomplete.

Technical logs may record actions without establishing their authority.

Repository histories may preserve modifications without explaining the decisions behind them.

AI-generated summaries may describe plausible development sequences without proving that those sequences occurred.

And a complete-looking traceability graph may contain unsupported or inferred relationships.

Recorded Development Activity ≠ Governed Development Traceability.

Task Traceability ≠ Constituent Action Traceability.

Recorded Association ≠ Governed Relationship.

Inferred Relationship ≠ Established Relationship.

Traceability-Supported ≠ Creation Legitimacy Established.

Traceable Software ≠ Correct Software.

DTM establishes the methodological conditions under which software creation

remains reconstructable across actors, methods, actions, tools, artifacts, and evidence.

It preserves the difference between a technical record and a supported

governance relationship.

It does not require every development process to use the same tools or generate identical logs.

It requires material creation relationships to remain identifiable,

evidentially supported, and continuous enough for competent governance assessment.

DTM transforms development records from disconnected technical

observations into governed creation traceability — ensuring that software can be created by humans, AI agents, and autonomous systems without losing the evidence-supported relationships necessary to reconstruct how, why, by whom or by what, and under which material conditions the software came into existence.